

Choosing the right Anthropic tool

Anthropic provides several ways to work with Claude. Start with the least technical option that can do the job, then move to a more powerful tool when you need repeatability, access to local files, or multiple steps.

Choosing the right model

Each participant has **\$100 in credits** for this workshop. **Claude Opus** should be a good balance of cost and ability for most hackathon tasks. Always review its output, especially scientific or clinical claims.

Choosing the right tool: A quick decision rule

- Choose [Claude chat](#) for one-off questions, explanations, document work, or small file tasks without coding.
- Choose **Claude Code** for local files, repositories, terminal commands, tests, and edits. It can run in your IDE, terminal, or [GitHub Codespaces](#).

Note: the workshop credits are API credits and do not cover Claude chat Pro/Max subscriptions.

Tool	Best for
Claude chat	One-off questions, writing, explanations, summaries, and small file transformations
Claude Code	Inspecting and changing repositories, running commands and tests, and working with local data. Use it in an IDE, terminal, or GitHub Codespaces.

Claude chat does not automatically have access to your computer. Claude Code can access the files and tools available in its workspace, but you still need to specify what you want changed.

Claude chat: simple tasks without technical setup

Use Claude chat when you want help with a self-contained task and do not need Claude to inspect a local project or run commands. Upload a document, spreadsheet, image, or small data file, describe the desired output, and review the result in the conversation.

Useful examples: - "Summarise this report in five bullet points and list anything that needs checking." - "Compare these two tables and identify rows that differ. Do not guess at missing values." - "Rewrite this explanation for a non-technical audience, keeping the scientific meaning unchanged."

Claude Code: inspect and change a codebase

Use Claude Code for repositories, local files, commands, tests, and multi-file changes. Start in the correct workspace and review proposed edits and scientific conclusions.

Claude Code quick-start pattern

Use this loop: **inspect -> plan -> implement -> test -> review**. Give Claude real files and a specific outcome, then verify what it changes.

1. Inspect

Ask Claude to examine the relevant files before proposing code. - "Inspect `data/example.tsv`. Describe its columns, missing values, and any inconsistent formatting. Do not edit it." - "Find where the report is generated. Explain the relevant files and call path."

2. Plan

Ask for a short plan that names the files and validation step. - "Before editing, give me a three-step plan and identify the test or command you will run."

3. Implement

Request the smallest change that meets the goal. - "Implement the parser in `scripts/parse.py`. Preserve the existing output format and add a test for an empty input."

4. Test

Ask Claude to run a focused check and show the result. - "Run the relevant test on the sample data, then try one malformed row and explain the behavior."

5. Review

Read the diff and check important claims yourself. - "Summarise the changes and list anything that still needs human or scientific review." - For genetics or clinical work: "Separate evidence from inference. Do not guess when the data is insufficient."

Fast tips

- **Show, don't describe:** give Claude a representative file or example.
- **Make one focused change at a time** and run a check after it.
- **Ask what is uncertain** and verify scientific or clinical claims independently.

Example

Inspect this repository and find where the Talos HTML report is generated. Do not modify anything yet. Explain which files are relevant to adding a case-summary panel.

Then ask Claude to implement the smallest version, run the relevant test or example command, and review the diff for unintended changes to Talos interpretation or output.